POLITÉCNICA
"Ingeniamos el futuro"

CAMPUS
DE EXCELENCIA
INTERNACIONAL

Front-end para Móviles (FEM)

Tema 2 – Interfaz de Usuario



miw.etsisi.upm.es

F. Javier Gil

Tema 2: Interfaz de usuario

- 2.1: Vistas: extienden la clase `View`
  - TextView, EditText, Button, Spinner, Radiobuttons, Checkboxes, ...
- 2.2: Layouts: extienden la clase `ViewGroup` y contienen otros controles (contenedores)
  - LinearLayout
  - FrameLayout
  - RelativeLayout
  - ListView, ...
- 2.3: Toasts, Snackbars, Menús, Diálogos, Notificaciones
- 2.4: Fragmentos

Interfaz de usuario: tamaño y unidades

- tamaño pantalla: densidades



- densidad (`dpi`): cantidad de pixels en un área física
  - `low`, `medium`, `high`, `extra-high`, `xxhigh`, `xxxhigh`
- density-independent pixel (`dp` o `dip`): dimensiones independientes de la densidad

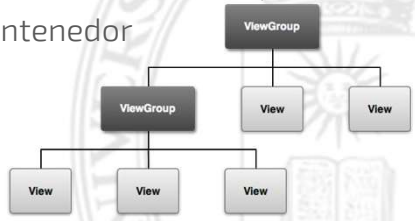
$$px = dp * (\frac{dpi}{160})$$

https://developer.android.com/guide/practices/screens_support.html

Interfaz de usuario: vistas y layouts

- `View`: bloque básico de los componentes de la interfaz de usuario
  - ocupa un área rectangular de la pantalla
  - responsable de la representación y manejo de eventos
  - ejemplos: botones, campos de texto, checkboxes, etc.
- `ViewGroup`: subclase básica para contenedores (*layouts*), define la estructura de la UI.
  - contenedores invisibles que contienen otros objetos `View`
  - define las propiedades del contenedor
  - ejemplos: listas, tablas, etc.

`Common Android Views`



Interfaz de usuario: interacción con vistas

1. especificando la acción en el atributo `onClick` de la vista:

```
<Button android:id="@+id/botonBorrar"
    android:onClick="accionBorrar" />
```

2. asignando una clase anónima a la vista que implementa la interfaz `onClickListener`:

```
bBorrar.setOnClickListener(new OnClickListener() {
        @Override
        public void onClick(View v) { ... }
);
```

3. implementando la interfaz `View.OnClickListener` en una clase:
   - debe implementar método `public void onClick(View v)`

Interfaz de usuario: TextView y EditText

- `TextView`
  - muestra texto al usuario (opcionalmente, permite su edición)
  - permitir seleccionar texto → `android:textIsSelectable`
  - especificación tipo entrada → `android:inputType`

`http://developer.android.com/reference/android/widget/TextView`

- `EditText`
  - subclase de `TextView` con métodos específicos para la edición de texto

`http://developer.android.com/reference/android/widget/EditText`

Interfaz de usuario: Button y Spinner

- `Button`
  - botón pulsador, subclase de `TextView`
  - se puede asignar método `setOnClickListener(…)`
  - o bien especificar atributo XML → `android:onClick`
  - atributos para la especificación del estilo

`http://developer.android.com/reference/android/widget/Button`

- `Spinner`
  - permiten seleccionar un elemento de una lista
  - método controlar selección → `onItemSelectedListener()`

`http://developer.android.com/reference/android/widget/Spinner`

Interfaz de usuario: RadioButton y Checkbox

- RadioButton
  - permiten elegir una única opción excluyente
  - se agrupan con el elemento `RadioGroup`
  - comportamiento se especifica con el atributo XML → `android:onClick`

`http://developer.android.com/reference/android/widget/RadioButton`

- Checkbox
  - botón específico que admite dos estados
  - atributo XML controlar selección → `android:onClick`

`http://developer.android.com/reference/android/widget/CheckBox`

Tema 2: Interfaz de usuario

- 2.1: Vistas: extienden la clase `View`
  - TextView, EditText, Button, Spinner, Radiobuttons, Checkboxes, ...
- 2.2: Layouts: extienden la clase `ViewGroup` y contienen otros controles (contenedores)
  - LinearLayout
  - FrameLayout
  - RelativeLayout
  - ListView, ...
- 2.3: Toasts, Snackbars, Menús, Diálogos, Notificaciones
- 2.4: Fragmentos

Interfaz de usuario: atributos comunes

- `width`, `layout_width`: especifica el ancho
  - dimension: preferiblemente `dp`
  - `match_parent`/~~`fill_parent`~~, `wrap_content`
- `heigth`, `layout_height`: especifica la altura
  - mismos valores que `width`
- `padding`: espacio entre el contenido y el borde del contenedor
- `layout_margin`: espacio entre vista y su contenedor
- `gravity`, `layout_gravity`: alineación de elementos
  - `left` , `center`, `right`, `center_vertical`, `center_horizontal`

Interfaz de usuario: LinearLayout

- alinea todos los elementos uno tras otro vertical u horizontalmente (propiedad `android:orientation`)
- propiedad `android:layout_weight`:
  - Permite asignar proporciones entre elementos contenidos
  - Si `orientation="horizontal"` → width a 0dp (match constraint)
    - Si `orientation="vertical"` → height a 0dp
- propiedad `android:weightSum` para modificar proporciones

http://developer.android.com/reference/android/widget/LinearLayout

Interfaz de usuario: FrameLayout

- se suele emplear como contenedor de un único elemento
- todos sus descendientes se alinean con la esquina superior izquierda (se superponen)
  - se pueden incluir varios descendientes y alinearlos con el atributo `android:layout_gravity`

http://developer.android.com/reference/android/widget/FrameLayout

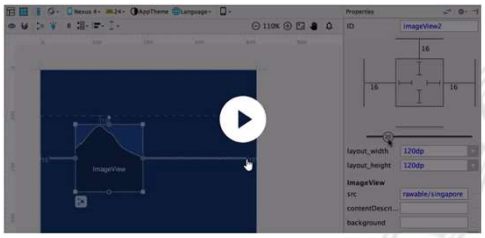
Interfaz de usuario: RelativeLayout

- la posición de los elementos puede establecerse en relación a la posición de otros elementos o la posición del padre
- algunos atributos:
  - `android:layout_toRightOf`
  - `android:layout_alignParentRight`
  - `android:layout_alignBottom`
  - `android:layout_centerInParent`
  - `...`

https://developer.android.com/reference/android/widget/RelativeLayout

Interfaz de usuario: ConstraintLayout

- layouts complejos sin anidamiento de vistas
- similar a `RelativeLayout`, pero más flexible
- cada vista debe tener al menos una restricción en horizontal y una en vertical



https://developer.android.com/develop/ui/views/layout/constraint-layout

Interfaz de usuario: ListView

- muestra una lista de elementos seleccionables
- se utilizará *scroll* si es necesario
- lista de elementos se rellena a partir de un adaptador (con datos de un array, una consulta a bases de datos, etc.)

```
ArrayAdapter<String> adaptador = new ArrayAdapter<>(this,
      android.R.layout.simple_list_item_1, misDatos);

ListView lvListado = findViewById(R.id.miListado);
lvListado.setAdapter(adaptador);
```

https://developer.android.com/reference/android/widget/ListView

Interfaz de usuario: otros contenedores

- TableLayout
- GridLayout
- ~~AbsoluteLayout~~ (*deprecated*)
- ScrollView
  - sólo puede contener un elemento (p.ej. otro contenedor)
- RecyclerView
- CoordinatorLayout
- ...

Tema 2: Interfaz de usuario

- 2.1: Vistas: extienden la clase `View`
  - TextView, EditText, Button, Spinner, Radiobuttons, Checkboxes, ...
- 2.2: Layouts: extienden la clase `ViewGroup` y contienen otros controles (contenedores)
  - LinearLayout
  - FrameLayout
  - RelativeLayout
  - ListView, ...
- 2.3: Toasts, Snackbars, Menús, Diálogos, Notificaciones
- 2.4: Fragmentos

Interfaz de usuario: Toast

- proporciona información sobre alguna operación en un popup
  - se muestra durante algunos segundos y después desaparece de manera automática
  - desaconsejados: preferible Snackbar



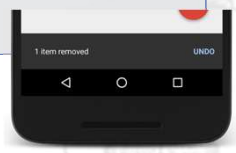

```
Toast.makeText(
   getApplicationContext(),
   "texto de la notificación",
   Toast.LENGTH_SHORT // or Toast.LENGTH_LONG
).show();
```

https://developer.android.com/guide/topics/ui/notifiers/toasts

Interfaz de usuario: Snackbar

- permite mostrar mensajes breves
- se muestra brevemente y después desaparece de manera automática
- se puede añadir una acción empleando el método `Snackbar.setAction(text, listener)`

```
Snackbar.make(
    getApplicationContext(),
    getString(R.string.message),
    Snackbar.LENGTH_SHORT // or Snackbar.LENGTH_LONG
).show();
```



https://developer.android.com/training/snackbar

Interfaz de usuario: Menús

- existen dos tipos de menús:
  - menús de opciones:
    - se puede añadir un recurso `/res/menu/filename.xml`
    - se deben implementar los métodos:
      - `public boolean onCreateOptionsMenu(Menu menu)`
      - `public boolean onOptionsItemSelected(MenuItem item)`

https://developer.android.com/guide/topics/resources/menu-resource.html

  - menús de contexto:
    - se registra el elemento con `registerForContextMenu(elementoSeleccionable)`
    - similar anteriores, `onCreateContextMenu()`, `onContextItemSelected()`

https://developer.android.com/guide/topics/ui/menus.html#context-menu

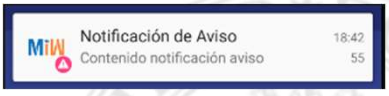
Interfaz de usuario: Diálogos

- se emplea la clase `DialogFragment` como contenedor. Controla eventos del ciclo de vida.
- tipos de diálogo:
  - **AlertDialog**: permite mostrar un *título*, un *área de contenido* con una lista de elementos seleccionables o un elemento personalizado y hasta tres *botones de acción*
  - **DatePickerDialog**/**TimePickerDialog**: elección de fecha/hora
  - ~~**ProgressDialog**~~ (desaconsejada, mejor ProgressBar)
  - **CharacterPickerDialog**: elección de caracteres

`https://developer.android.com/guide/topics/ui/dialogs`

Interfaz de usuario: Notificaciones

- mensajes de la aplicación que se muestran fuera de la interfaz de usuario de la aplicación
- se crea un objeto empleando el objeto `NotificationCompat.Builder`
- una notificación debe contener:
  - icono pequeño (`setSmallIcon()`)
  - título (`setContentTitle()`)
  - texto detallado (`setContentText()`)



- la notificación se muestra empleando el método `notify()` del servicio de notificaciones

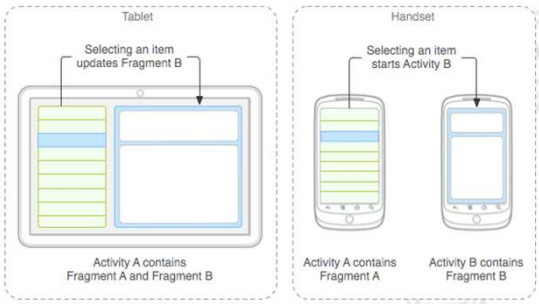
`https://developer.android.com/guide/topics/ui/notifiers/notifications`

Tema 2: Interfaz de usuario

- 2.1: Vistas: extienden la clase `View`
  - TextView, EditText, Button, Spinner, Radiobuttons, Checkboxes, ...
- 2.2: Layouts: extienden la clase `ViewGroup` y contienen otros controles (contenedores)
  - LinearLayout
  - FrameLayout
  - RelativeLayout
  - ListView, ...
- 2.3: Toasts, Snackbars, Menús, Diálogos, Notificaciones
- 2.4: Fragmentos

Interfaz de usuario: Fragmentos

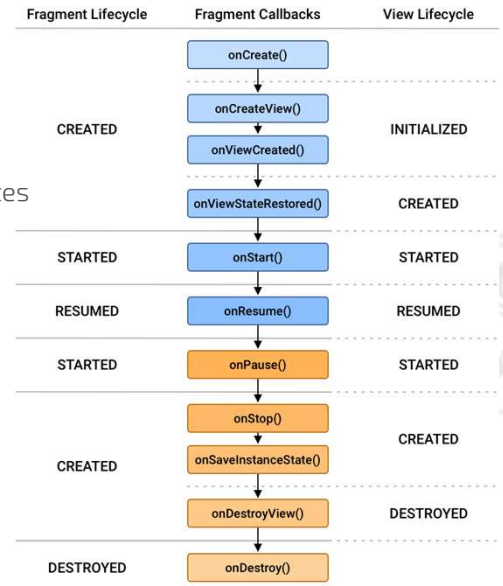
- contenedor dirigido hacia dispositivos "grandes"
- define el comportamiento de la interfaz de usuario de una actividad
  - dinámicos: puede ser añadidos, sustituidos o eliminados
- ciclo de vida propio

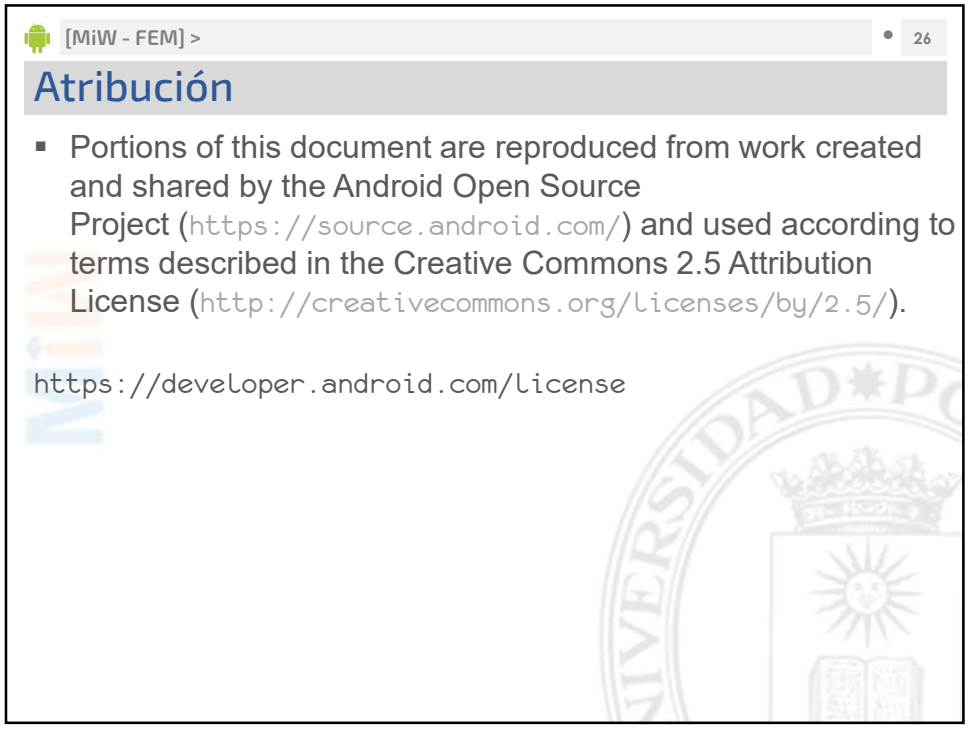


http://developer.android.com/guide/components/fragments

Interfaz de usuario: Fragmentos

- subclase de `Fragment`
- ciclo vida propio
- métodos obligatorios:
  - `onCreate()`
    inicialización componentes
  - `onCreateView()`
    diseña la UI fragmento
  - `onPause()`
    confirmación cambios
- algunas subclases
  - `DialogFragment`
  - `ListFragment`
  - `PreferenceFragment`




[MiW - FEM] >
26
Atribución
Portions of this document are reproduced from work created and shared by the Android Open Source Project (https://source.android.com/) and used according to terms described in the Creative Commons 2.5 Attribution License (http://creativecommons.org/licenses/by/2.5/).
https://developer.android.com/license